

《智能体云原生开发》期末大作业

一、摘要

1. 我们小组选择**命题一**，旨在开发一个基于大模型的 PPT 智能扩展与辅助学习 Agent。我们实现了一个完整的云原生智能体系统，核心功能摘要如下：
- 1. **PPT 深度语义解析**：基于 `python-pptx` 实现了对 PPT 文件的结构化提取，能够准确识别标题、正文及层级关系。
 - 2. **智能知识扩展 (RAG + Agent)**：利用 `LangGraph` 构建了具备“检索-决策-生成-校验”能力的智能体 workflow。系统能自动判断知识点是否需要外部搜索 (Arxiv/Semantic Scholar)，并利用向量数据库 (`Milvus`) 进行检索增强生成。
 - 3. **个性化学习仪表盘**：集成用户画像与学习路径规划功能，支持设定学习目标、生成每日任务清单，并基于用户兴趣推荐相关学术资源，打造“伴随式”学习体验。
 - 4. **云原生微服务架构**：系统完全容器化，后端服务、Redis 消息队列、`Milvus` 向量库均通过 `Docker Compose` 编排，具备良好的扩展性与部署便捷性。
 - 5. **全栈交互体验**：提供了基于 `React` 的前端界面与 `FastAPI` 后端服务，支持文件上传、异步任务处理及生成结果的实时展示。

团队分工 (贡献度均为33.3%)

- **韩悦 (10213903418)：智能体架构与编排**
 - **负责模块**：`LangGraph` 工作流、PPT 基础结构解析、PPT 图片解析 (OCR)。
 - **核心贡献**：设计并实现了“检索-决策-生成-校验”的闭环工作流；优化了 Prompt 工程以提升 Agent 的语义理解与容错能力；利用 `python-pptx`，`PyMuPDF` 与 OCR 技术解决了 PPT、PDF 及 Word 等多模态数据的提取难题。

Commits

hy

All usersAll time

Commits on Jan 19, 2026

imageOCR

alei committed 20 hours ago

448eb21

Commits on Jan 18, 2026

first version

alei committed 2 days ago

6e82460

Commits

hy_version1

All usersAll time

Commits on Jan 20, 2026

Delete test_api.py

Jerry-yung authored 2 hours ago

Verified42887bd

修复资源推荐并完成LangGraph

alei committed 4 hours ago

7527d1e

- **贾馨雨 (10235501437)：全栈业务与个性化**
 - **负责模块**：用户注册登录、学习目标控制、个性化推荐系统、前端开发。
 - **核心贡献**：构建了完整的用户认证体系；实现了基于用户画像的个性化学习计划生成算法；完成了 `React` 前端与后端 API 的对接；撰写**实验报告**。

Commits

jxy

All usersAll time

Commits on Jan 19, 2026

docs: Update README with new features (learning plan, user center) and UI optimizations xiaoxinyusss committed yesterday	9f9da82		<>
feat: Fix plan saving/sync and improve UI scaling xiaoxinyusss committed yesterday	2021e51		<>
Revert recommended resources UI to previous version while keeping plan UI improvements xiaoxinyusss committed yesterday	4b81635		<>
Fix undefined daily tasks: enhance frontend rendering and backend prompt schema xiaoxinyusss committed yesterday	f5d194e		<>
Update project with full features: auth, localization, search, and fixes xiaoxinyusss committed yesterday	bcd1b4b6		<>

- 杨云天（10245501405）：数据检索与多模态解析
 - 负责模块：多源搜索 API、参考文件解析 (PDF/Word)。
 - 核心贡献：封装了 Arxiv/Semantic Scholar 等多源搜索接口并实现结果融合；录制demo视频

Commits

yyt

All usersAll time

Commits on Jan 20, 2026

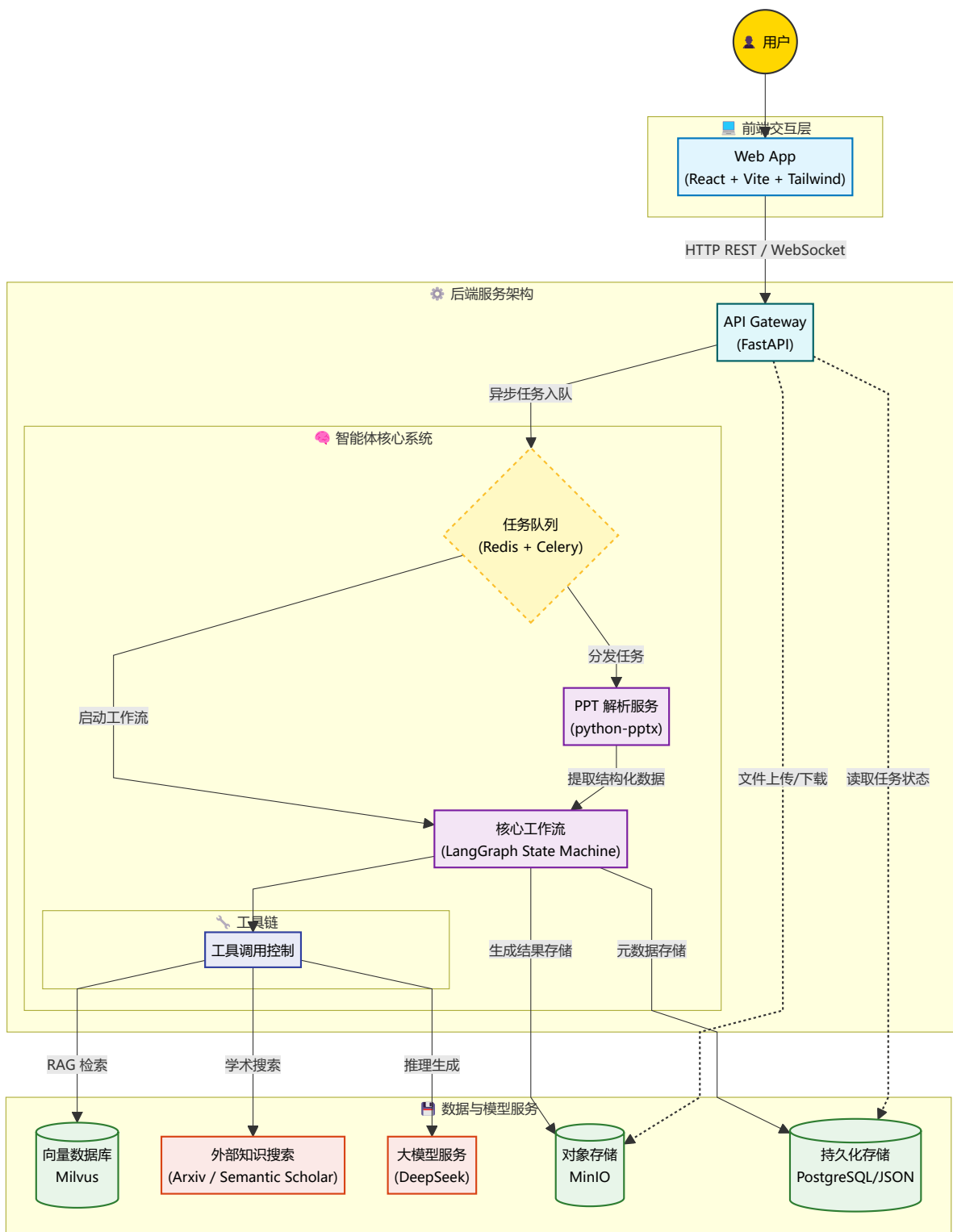
Update README.md Jerry-yung authored 17 hours ago	Verifiedfc410f1		<>
功能合并 Jerry-yung committed 17 hours ago	3c65e34		<>

Commits on Jan 19, 2026

feat: 添加参考文件上传、LLM关键词提取、内容过滤等新功能 Jerry-yung committed yesterday	adb6fd2		<>
更新 README.md: 添加最新功能说明 (LLM关键词提取、内容过滤、Wikipedia禁用等) Jerry-yung committed yesterday	b612170		<>

二、架构设计

2.1 系统架构图与数据流向



数据流向说明:

- 用户操作:** 用户通过前端上传PPT并触发扩展任务。
- API 接收:** API网关接收请求，创建异步任务并存入Redis队列。
- 任务分发:** Celery Worker (集成在 Agent 服务中) 消费任务，启动解析服务提取PPT文本和结构。
- 智能编排:** 解析后的结构化数据送入LangGraph智能体工作流。
- 工具调用:** 智能体根据策略，依次调用向量检索 (RAG)、外部搜索和LLM生成等工具。
- 结果存储:** 最终生成的扩展内容与原始PPT关联，存储于 MinIO 和 本地数据文件 (生产环境可迁移至 PostgreSQL)。
- 前端展示:** 前端通过轮询获取任务结果并展示。

2.2 云原生组件清单

组件	版本	用途	容器名	命题对应与考核点
Docker	24+	应用容器化与编排基石	-	云原生核心 ：实现环境一致性，避免“单一脚本运行”。
Redis	7-alpine	1) Celery任务队列 Broker 2) 用户会话缓存 3) Agent中间状态缓存	ppt-agent-redis	容错与性能 ：异步解耦，提升系统响应与稳定性。
Milvus	v2.3.x	存储PPT切片及扩展知识的向量，实现语义相关性检索	ppt-agent-milvus	命题核心 ：实现基于语义的RAG，支撑“联想内容相关度”考核。
MinIO	RELEASE	对象存储，用于存放用户上传的原始PPT文件和生成的扩展文档	ppt-agent-minio	云原生存储实践 ：分离数据与计算。
EtcD	v3.5.x	Milvus的元数据存储依赖	ppt-agent-etcd	支撑分布式向量数据库的运行。
Persistence	-	用户数据与任务状态存储	(Internal)	数据持久化 ：当前使用JSON 文件存储，设计上支持 PostgreSQL 扩展。

2.3 LLM Agent 工具链

- **核心框架**：langchain, langgraph (工作流编排), langchain-openai (兼容DeepSeek API)
- **文档解析**：python-pptx (PPTX), PyMuPDF (PDF参考文档), python-docx (Docx参考文档)
- **文本处理与向量化**：sentence-transformers (all-MiniLM-L6-v2 生成Embedding), pymilvus (向量检索客户端)
- **外部知识搜索**：arxiv (学术论文), Semantic Scholar (学术搜索), Crossref (学术文献), openAlex (开放学术图谱)
- **其他工具**：pytesseract (OCR), requests (API调用)

三、核心功能与 workflow

3.1 用户操作流程

1. **上传与分析**：用户上传PPT文件。系统解析并展示其大纲结构（目录、标题、页面预览）。
2. **选择与配置**：用户选择需要扩展的特定页面或全部页面，可选择附加参考文档（PDF/Docx）。
3. **触发扩展**：用户点击“智能扩展”，系统创建异步任务。

4. **异步处理**：后端智能体按工作流（见下文智能体策略）进行处理，用户可在任务中心查看实时状态。
5. **查看与导出**：处理完成后，用户可在交互式界面中通过分栏（背景、原理、公式、示例、总结）查看生成的扩展内容，并支持导出为 Markdown 或 PDF 格式。
6. **导出**：将扩展后的完整内容导出为新的PPTX或Markdown文件。

3.2 功能模块详解

语义解析模块

- **输入**：`.pptx` 文件。
- **输出**：结构化JSON，包含 `slides[]` -> `title`, `body_text`, `shapes`（类型、位置、文本），`notes`。
- **挑战应对**：处理复杂布局、SmartArt、图表标题的提取。

知识扩充模块

- **分块与向量化**：将解析出的文本按语义分块，通过 `sentence-transformers` 生成向量，存入 Milvus。
- **检索**：用户查询时，将当前幻灯片标题/内容向量化，在 Milvus 中进行相似性检索，返回最相关的 k 个知识块。
- **生成**：将原始内容、检索到的相关上下文、外部搜索结果（基于 DecideSearch 节点的决策结果）组合成 Prompt，送入 LLM 生成扩展内容。

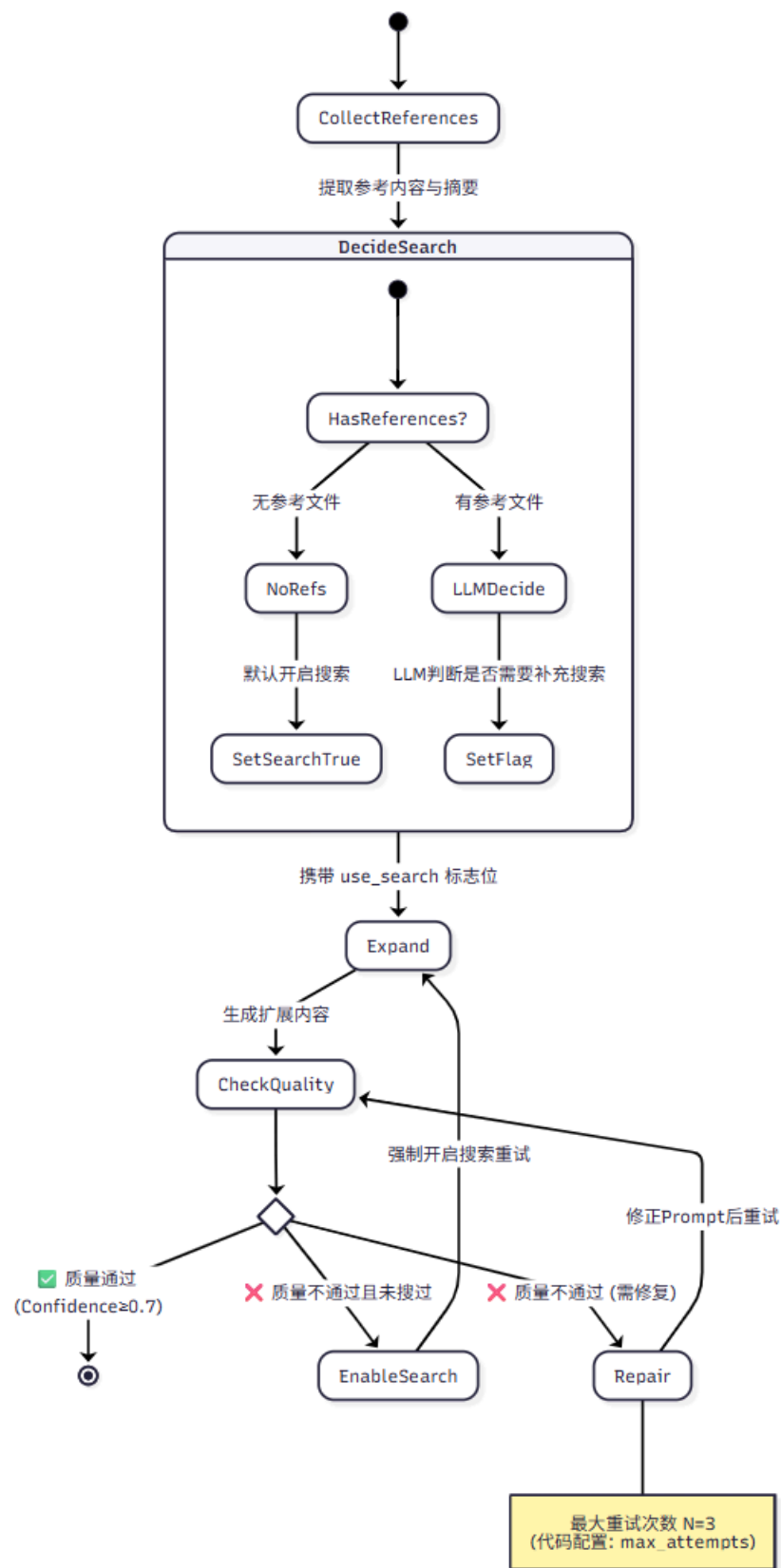
多维搜索模块

- **策略**：由智能体决策节点判断是否需要及调用哪个外部搜索工具。
- **结果处理**：对搜索结果进行摘要、去重和可信度标注。

四、智能体策略

遵循“规划(Plan) -> 执行(Act) -> 校验(Check) -> 修复(Repair)”的闭环，内置容错。

4.1 LangGraph workflows 设计



节点详解:

- **DecideSearch (决策节点):**
 - **Prompt目标:** 判断当前PPT知识点上下文是否足够进行高质量扩展。
 - **输入:** 幻灯片标题、正文、已有的参考文档摘要。
 - **输出:** `{"use_search": boolean, "reason": str}`。

- 注：此节点仅设置标志位，实际搜索动作由 `Expand` 节点内部服务执行。
- **Expand (扩展生成节点):**
 - **输入：**原始内容 + 参考文档片段 + 搜索结果（如果有）。
 - **核心Prompt：**严格遵循结构化输出要求 (JSON)，强调基于给定信息生成，减少幻觉。
 - 注：若 `use_search` 为 `True`，此节点会先调用 `SearchService` 执行多源搜索。
- **CheckQuality (校验层节点):**
 - **目的：**实现“注重容错”要求，对抗模型幻觉。
 - **检查项：**事实一致性、格式合规性、内容完整性、与原始主题的相关性。
 - **“低温”LLM：**使用更低温度 (`temperature=0.1`) 的LLM进行严谨性检查。
- **Repair (修复节点):**
 - **策略：**根据CheckQuality返回的具体错误信息，重构或修正Prompt，直接生成修复后的内容，并再次送入 CheckQuality 节点。

4.2 关键Prompt模板

```
# 1. 搜索决策Prompt (对应DecideSearch节点)
def get_search_decision_prompt(title, content, context="无",
reference_excerpt="无"):
    return f"""你是一位“学习资料检索策略”的决策助手。现在系统将为PPT知识点生成扩展内容。
你需要决定：在已经有“用户参考文件”的情况下，是否还需要外部检索（Arxiv / Semantic Scholar /
Crossref / OpenAlex）。

**标题**： {title}
**原始内容**：
{content}
**上下文信息**：
{context}
**用户参考文件摘录**：
{reference_excerpt}

**决策规则**：
1. 如果参考文件已充分覆盖核心概念、定义、公式、步骤：可以不进行外部检索（use_search=false）。
2. 如果参考文件内容不够、含糊、缺少来源、或可能需要更权威论文/定义支撑：应进行外部检索
（use_search=true）。
3. 若参考文件与主题相关性弱：use_search=true。
4. 你只做决策，不要输出解释性长文。

**输出格式**（必须严格 JSON，不要 Markdown）：
{{
  "use_search": true/false,
  "reason": "一句话原因（<=30字）"
}}
"""

# 2. 校验层Prompt (对应CheckQuality节点)
def get_check_layer_prompt(original, expansion_json, reference_excerpt="无",
used_search=False):
    return f"""你是一位专业的“内容校验(Check Layer)审核员”。你要严格检查扩展内容是否存在幻
觉、错误或不一致。

**原始内容**：
```

```

{original}

**扩展内容(JSON)**:
{expansion_json}

**用户参考文件摘录**:
{reference_excerpt}

**是否已使用外部检索**: {"是" if used_search else "否"}

**审核标准**:
1. 语义相关性: 扩展是否围绕原始内容展开?
2. 事实准确性: 是否出现明显错误、编造术语/论文/结论?
3. 一致性: 内部是否自相矛盾? 是否与参考文件冲突?
4. 可验证性: 如果给出“引用/论文/链接”, 是否看起来合理?

**输出格式** (必须严格 JSON, 不要 Markdown):
{{
    "is_relevant": true/false,
    "is_accurate": true/false,
    "is_consistent": true/false,
    "confidence": 0.0,
    "issues": ["问题1", "问题2"]
}}

**阈值**:
- confidence >= 0.7 且三个 is_* 均为 true 才算通过
"""

# 3. 知识扩展Prompt (对应Expand节点, 带参考文件的版本)
def get_expansion_prompt_with_reference_files(title, content, context="无",
reference_contents=None, search_results=None):
    return f"""你是一位专业的教育内容扩展助手, 擅长将简短的知识点扩展为详细的学习材料。**请优先使用用户提供的参考文件内容**, 结合外部权威资料为PPT知识点生成详细的扩展内容。

**标题**: {title}
**原始内容**: {content}
**上下文信息**:
{context}
**用户提供的参考文件** (优先使用):
{{reference_text}}
**外部权威参考资料** (作为补充):
{{references_text}}

**扩展要求**:
1. **背景说明**: 解释知识点的来龙去脉... **优先参考用户提供的参考文件中的信息**
2. **原理阐述**: 详细阐述核心原理... **优先结合参考文件中的权威观点**
3. **公式推导**: 如有数学公式, 请提供完整的推导过程 (使用LaTeX格式, 如  $E=mc^2$ )
4. **代码示例**: 如有编程实现, 请提供Python代码示例, 并添加详细注释
5. **要点总结**: 总结核心要点, 便于快速复习

**输出格式** (必须严格按照以下JSON格式输出):
```json
{{
 "background": "背景说明内容...",
 "principles": "原理阐述内容...",

```



```
"formulas": "公式推导内容（如适用）",
"examples": "代码示例内容（如适用）",
"summary": "要点总结内容..."
}}
```

## 五、系统部署与运行指南

### 5.1 快速开始

```
后端服务 (Docker Compose)

```bash
# 1. 克隆代码
git clone -b hy_version1 https://github.com/alei2024/ppt-extension-agent.git ppt-extension-agent-hy_version1
cd ppt-extension-agent-hy_version1

# 2. 配置环境变量
cp env.example .env
# 编辑 .env，填入 LLM_API_KEY 等必要信息

# 3. 启动后端核心服务 (API, Redis, Milvus, MinIO)
docker-compose up -d

# 4. 验证后端状态
# API文档: http://localhost:8000/docs

# 4. 验证后端状态
# API文档: http://localhost:8000/docs
```

前端应用 (React + Vite)

```bash
# 1. 进入前端目录
cd frontend

# 2. 安装依赖
npm install

# 3. 启动开发服务器
npm run dev

# 4. 访问应用
# 地址: http://localhost:3000 (默认配置了代理转发到后端8000端口)
```

5.2 项目目录结构

```
├─ ppt-extension-agent-hy_version1/
│   └─ agents/           # LangGraph智能体工作流 (ppt_agent.py)
│   └─ api/              # 路由端点 (routes.py, auth_routes.py,
learning_routes.py)
│   └─ app/              # 应用配置与入口 (config.py, main.py)
│   └─ frontend/         # React前端项目
```

```

|   |   └─ src/
|   |       └─ components/    # 核心组件库
|   |           └─ Auth/      # 认证组件 (Login/Register)
|   |           └─ Dashboard/  # 任务看板
|   |           └─ ExpansionPanel/ # 扩展内容展示 (分栏视图/导出)
|   |           └─ PPTViewer/   # PPT结构化预览
|   |           └─ ReferenceFileUpload/ # 参考文件上传
|   |       └─ services/      # 前端API服务
|   └─ logs/                  # 运行日志
|   └─ output/                # 生成结果输出
|   └─ services/              # 业务逻辑服务
|       └─ parser.py          # PPT解析核心
|       └─ knowledge_expander.py # 知识扩展生成器
|       └─ search_service.py    # 多源外部搜索
|       └─ reference_parser.py  # 参考文件解析与相关性判断
|       └─ image_recognizer.py  # 图片OCR与描述生成
|       └─ export_service.py    # 结果导出 (Markdown/PPTX)
|       └─ user_service.py      # 用户管理
|   └─ uploads/                # 文件上传目录
|   └─ utils/                  # 工具函数 (prompts.py, vector_db.py,
llm_factory.py)
|   └─ docker-compose.yml      # 多服务编排定义
|   └─ Dockerfile              # 后端容器构建文件
|   └─ env.example              # 环境变量示例
|   └─ requirements.txt         # Python依赖列表
└─ TECHNICAL_DOCS.md          # 本技术文档

```

5.3 核心配置说明 (.env)

```

# 服务端配置
APP_NAME="PPT Extension Agent"
DEBUG=True

# OpenAI / DeepSeek 配置 (通过 SiliconFlow 或 DeepSeek 官方)
OPENAI_API_KEY=sk-xxxxxx
OPENAI_BASE_URL=https://api.siliconflow.cn/v1

# 向量数据库 (Milvus)
MILVUS_HOST=milvus-standalone
MILVUS_PORT=19530

# Redis 配置
REDIS_HOST=redis
REDIS_PORT=6379

# 对象存储 (MinIO)
MINIO_ENDPOINT=minio:9000
MINIO_ACCESS_KEY=minioadmin
MINIO_SECRET_KEY=minioadmin

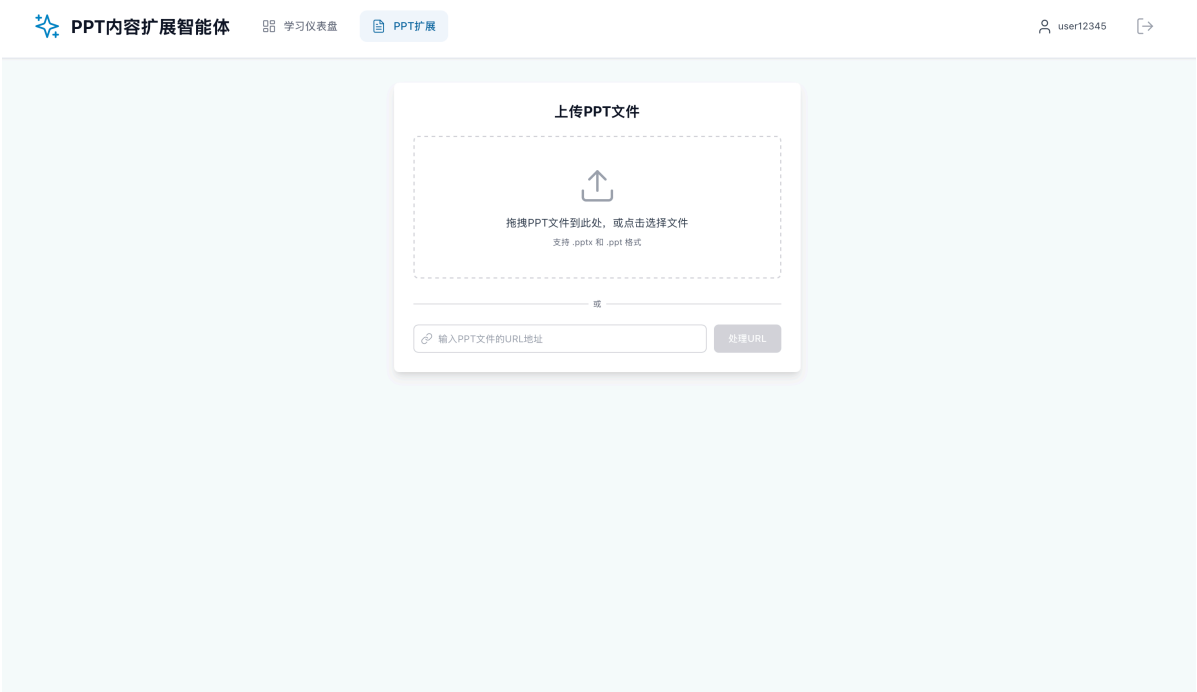
```

六、项目展示截图

6.1 用户注册登录页面



6.2 PPT 上传页面及上传结果示例



ce904921-101d-4aff-81a7-d8ae9337ed49_机器学习.pptx

重新上传

强化学习：AlphaGo通过与自己对弈不断学习，在与李世石的围棋比赛中，运用强化学习策略取得胜利，展现了AI的决策能力。

基本概念

监督学习：如垃圾邮件识别，系统通过标记好的邮件数据学习，当收到新邮件时，可判断其是否为垃圾邮件，像Gmail的垃圾邮件过滤功能。

无监督学习：电商平台对用户购物数据聚类分析，如亚马逊将购买习惯相似的用户分群，实现商品精准推荐，提升用户购买率。

强化学习：AlphaGo通过与自己对弈不断学习，在与李世石的围棋比赛中，运用强化学习策略取得胜利，展现了AI的决策能力。

图片内容



描述：这是一张机器学习或数据科学项目的流程图，展示了从原始数据到模型部署的完整工作流程。主要步骤包括数据获取与处理、特征工程、模型选择与训练、评估优化以及最终的应用部署。图片类型为流程图或示意图，在PPT中用于清晰地说明数据建模项目的标准化步骤和生命周期，帮助观众理解各个环节的逻辑顺序与关键任务。

识别文字： 展开

✓ 已选中

导出选项

Markdown

PDF

背景说明 原理阐述 公式推导 代码示例 重点总结

要点总结

1. **核心定义**：强化学习是智能体通过与环境交互，以“试错”方式学习最大化累积奖励策略的机器学习范式。2. **核心框架**：基于马尔可夫决策过程（MDP），包含状态、动作、奖励、策略和价值函数等关键概念。3. **AlphaGo的关键创新**：并非单纯使用强化学习，而是开创性地结合了监督学习（从人类棋谱学习初始策略）、强化学习（通过自我对弈优化策略和价值函数）和蒙特卡洛树搜索（实现高效的实时决策搜索）。4. **学习机制**：使用策略梯度等算法，根据整体回合的奖励信号（如棋局的胜负）来调整策略网络的参数，使能带来胜利的决策序列概率增大。5. **意义与影响**：AlphaGo的胜利证明了强化学习在解决超高复杂度、完美信息博弈问题上的强大能力，其“从交互中学习”的范式对游戏AI、机器人、自动驾驶、资源管理等序列决策领域产生了深远影响。

参考文献 (3篇)

1. Mastering the game of Go with deep neural networks and tree search

Nature

2. Reinforcement Learning: An Introduction (2nd Edition)

Sutton & Barto

3. OpenAI Gym: A toolkit for developing and comparing reinforcement learning algorithms

OpenAI

ce904921-101d-4aff-81a7-d8ae9337ed49_机器学习.pptx

重新上传

强化学习：AlphaGo通过与自己对弈不断学习，在与李世石的围棋比赛中，运用强化学习策略取得胜利，展现了AI的决策能力。

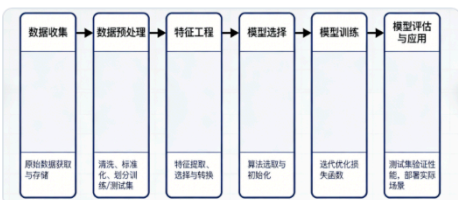
基本概念

监督学习：如垃圾邮件识别，系统通过标记好的邮件数据学习，当收到新邮件时，可判断其是否为垃圾邮件，像Gmail的垃圾邮件过滤功能。

无监督学习：电商平台对用户购物数据聚类分析，如亚马逊将购买习惯相似的用户分群，实现商品精准推荐，提升用户购买率。

强化学习：AlphaGo通过与自己对弈不断学习，在与李世石的围棋比赛中，运用强化学习策略取得胜利，展现了AI的决策能力。

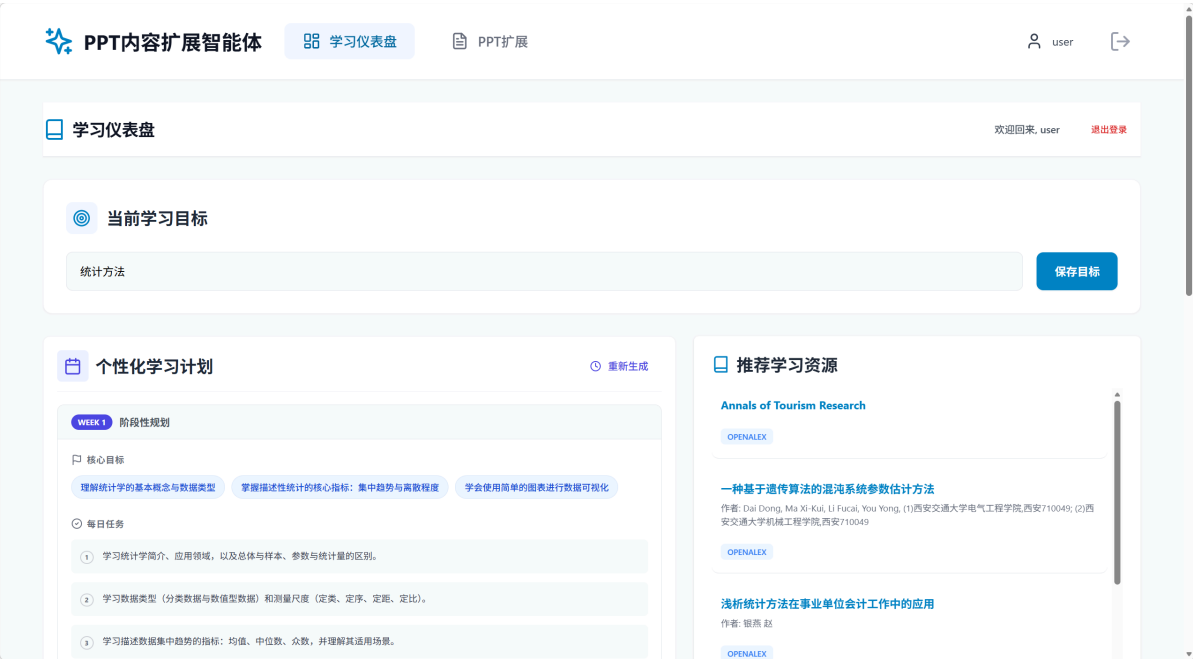
图片内容



描述：这是一张机器学习或数据科学项目的流程图，展示了从原始数据到模型部署的完整工作流程。主要步骤包括数据获取与处理、特征工程、模型选择与训练、评估优化以及最终的应用部署。图片类型为流程图或示意图，在PPT中用于清晰地说明数据建模项目的标准化步骤和生命周期，帮助观众理解各个环节的逻辑顺序与关键任务。

识别文字： 展开

6.3 用户设定学习目标及个性化计划生成



6.4 用户个人主页及历史学习计划

